

Data Analysis and Visualization

PART II:

Advanced Visualization and Creating Dashboards

Lecturer: Diu Tran, PhD

Department of Computer Science
Faculty of Information Technology
Ton Duc Thang University

Ho Chi Minh, Week 4, 2025

Content

- 1 Revise week 3
- 2 Creating Dashboards with Plotly
- 3 Creating Dashboards with Dash
- 4 Lab 4

Revise week 3

Advanced Data Visualization

- Waffle Charts

`WaffleCharts_WordClouds_RegressionPlots.ipynb`

- Word Cloud

`WaffleCharts_WordClouds_RegressionPlots.ipynb`

- Choropleth Maps

`Creating_maps_visualizing_geospat.ipynb`

- Treemaps and Pivot Charts

`Treemaps_and_Pivot_Charts.ipynb`

Create dashboards with Plotly

- Dashboarding Motivation
- Introduction to Plotly
- Using Plotly library to create Scatter, Line, Bar, Bubble, Histogram, Pie, Sunburst.

Dashboarding motivation

- Real-time visuals simplify business moving parts
- Display Key Performance Indicators (KPI) for analysis
- Help businesses by providing the big picture

The best dashboards answer critical business questions

Scenario

Review flight performance of US domestic flights

Yearly report items:

- Top 10 airline carriers in the year 2019
- Number of flights in 2019
- Number of travelers from California state

Report type 1: without dashboard

1.

Reporting_Airline	Flights
WN	13972
DL	10241
AA	9853
OO	8515
UA	6430
YX	3416
MQ	3368
B6	3170
OH	2969
AS	2783

2.

Month	Flights
1	6125
2	5578
3	6553
4	6282
5	6441
6	6583
7	6798
8	6753
9	6140
10	6533
11	6280
12	6550

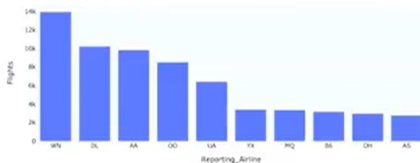
3.

DestState	DistanceGroup	Flights
AK	10	4
AL	8	1
AR	7	1
AZ	2	361
AZ	3	220
AZ	4	9
CA	1	263
CA	2	2256
CA	3	36
CO	3	19
CO	4	401
CO	5	2
CT	11	2
FL	9	51
FL	10	90
FL	11	26
GA	8	114
GA	9	65
HI	10	168
HI	11	116
ID	2	7
ID	3	60
IL	7	169
IL	8	154
IN	8	16
KY	8	13
KY	9	5
LA	7	33
LA	8	14
MA	11	128

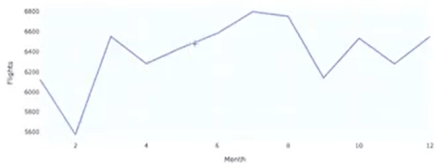
Report type 2: with dashboard

Airline Dashboard

Top 10 airline carrier in year 2019 in terms of number of flights



Number of flights by month - 2019



Number of travelers from CA to other states split by distance group



Python dashboarding tools: <https://pyviz.org/dashboarding/>

Introduction to Plotly library

- Plotly is an interactive, open-source plotting library
- Supports over 40 unique chart types
- Includes various types of charts
- Visualizations can be:
 - ▶ Displayed in Jupyter Notebook
 - ▶ Saved to HTML files
 - ▶ Used in developing Python-built web applications

Plotly: two sub-modules

- Plotly Graph Objects is the low-level interface to figures, traces and layout
- Plotly Express is a high level wrapper for Plotly. It uses graph objects internally.

File: `Plotly_Basics.ipynb`

Additional Resources for Plotly

Using Plotly to create dashboards:

- Plotly python:
<https://plotly.com/python/getting-started/>
- Plotly graph objects with example:
<https://plotly.com/python/graph-objects/>
- Plotly express:
<https://plotly.com/python/plotly-express/>
- API reference:
<https://plotly.com/python-api-reference/>

Additional useful resources:

- Plotly cheatsheet:
https://images.plot.ly/plotly-documentation/images/plotly_js_cheat_sheet.pdf
- Plotly community:
<https://community.plotly.com/c/plotly-python/5>
- Related blogs: <https://plotlygraphs.medium.com/>

Introduction to Dash

Dash - an overview

- Dash is an open-source User Interface Python library from Plotly
- Easy to build Graphical User Interface
- Dash is declarative and reactive
- Rendered in a web browser and can be deployed to servers
- Inherently mobile and cross-platform-ready

Dash components

- Core components

- ▶ `import dash_core_components as dcc`
- ▶ Higher-level interactive components generated with JavaScript, HTML, and CSS through the React.js library
- ▶ Examples: creating a slider, input area, check items, date picker, and other components

- HTML components

- ▶ `import dash_html_components as html`
- ▶ Provides classes for all HTML tags and the
- ▶ Keyword arguments describe the HTML attributes like style, class name, and ID.

Additional Resources for Dash

- Complete Dash user guide: <https://dash.plotly.com/>
- Dash core components:
<https://dash.plotly.com/dash-core-components>
- Dash HTML components:
<https://dash.plotly.com/dash-html-components>
- Dash community forum:
<https://community.plotly.com/c/python/25>
- Related blogs: <https://medium.com/plotly/tagged/dash>

Make dashboards interactive

Describe the Callback function and determine how to connect core and HTML components using Callbacks

- A callback function is a Python function
- It is automatically called by Dash
- The `@app.callback` decorator decorates the callback function in order to tell Dash to call it whenever there's a change in the input component value.
- The callback function takes input and output components as parameters and performs operations to return the desired result for the output component.

Dash-callback function

```
def callback_function:  
    .....  
    .....  
    .....  
    return some_result  
  
@app.callback(Output, Input)
```

Two parameters:

- Input: It's provided to the callback function
- Output: It sets results returned from the callback

Callback with one input

```
# Import required packages
import pandas as pd
import plotly.express as px
import dash
import dash_html_components as html
import dash_core_components as dcc
from dash.dependencies import Input, Output

# Read the data
airline_data = pd.read_csv('airline_2m.csv',
                           encoding = "ISO-8859-1",
                           dtype={'Div1Airport': str,
                                  'Div1TailNum': str,
                                  'Div2Airport': str,
                                  'Div2TailNum': str})
```

Callback with one input

```
app = dash.Dash()

# Design dash app layout
app.layout = html.Div(children=[ html.H1('Airline Dashboard',
                                         style={'textAlign': 'center', 'color': colors['text'],
                                         'font-size': 40}),

                                html.Div(["Input: ", dcc.Input(id='input-yr', value='2010',
                                                                type='number', style={'height': '50px', 'font-size': 35}),],
                                         style={'font-size': 40}),

                                html.Br(),
                                html.Br(),
                                html.Div(dcc.Graph(id='bar-plot')),

                                ])

```

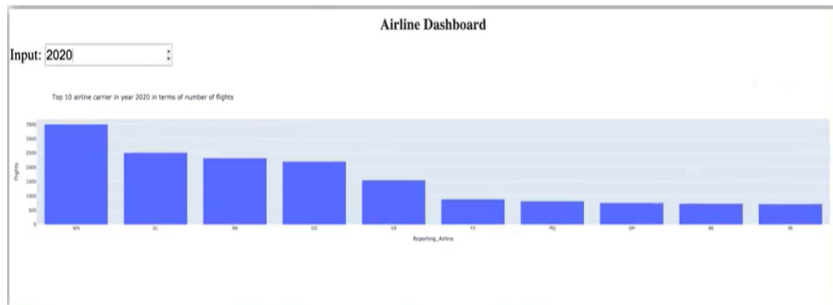
Callback with one input

```
@app.callback( Output(component_id='bar-plot', component_property='figure'),
               Input(component_id='input-yr', component_property='value'))

def get_graph(entered_year):
    # Select data
    df = airline_data[airline_data['Year']==int(entered_year)]
    # Top 10 airline carrier in terms of number of flights
    g1 = df.groupby(['Reporting_Airline'])['Flights'].sum().nlargest(10).reset_index()
    # Plot the graph
    fig1 = px.bar(g1, x='Reporting_Airline', y='Flights', title='Top 10 airline carrier in
                    year ' + str(entered_year) + ' in terms of number of flights')
    fig1.update_layout()
    return fig1

if __name__ == '__main__':
    app.run_server(port=8002, host='127.0.0.1', debug=True)
```

Callback with one input



Callback with two inputs

```
app = dash.Dash()
# Design dash app layout
app.layout = html.Div(children=[ html.H1('Airline Dashboard', style={'textAlign': 'center',
                                'color': colors['text'], 'font-size': 40}),
                                html.Div(["Year: ", dcc.Input(id='input-yr', value='2010',
                                type='number', style={'height': '50px', 'font-size': 35}),
                                ], style={'font-size': 40}),
                                html.Div(["State Abbreviation: ", dcc.Input(id='input-ab',
                                value='AL', type='text', style={'height': '50px',
                                'font-size': 35})]), style={'font-size': 40}),
                                html.Br(),
                                html.Br(),
                                html.Div(dcc.Graph(id='bar-plot')),
                                ])
```

Callback with two inputs

```
@app.callback( Output(component_id='bar-plot', component_property='figure'),
               [ Input(component_id='input-yr', component_property='value'),
                 Input(component_id='input-ab', component_property='value')])

def get_graph(entered_year, entered_state):
    # Select data
    df = airline_data[ (airline_data['Year']==int(entered_year)) &
                      (airline_data['OriginState'] == entered_state)]

    # Top 10 airline carrier in terms of number of flights
    *****

    fig1.update_layout()
    return fig1

if __name__ == '__main__':
    app.run_server(port=8002, host='127.0.0.1', debug=True)
```

Additional resources for interactive dashboards

To learn more about making interactive dashboards in Dash:

- Python decorators reference 1:
<https://realpython.com/primer-on-python-decorators/>
- Python decorators reference 2:
<https://peps.python.org/pep-0318/#current-syntax>
- Callbacks with example:
<https://dash.plotly.com/basic-callbacks>
- Dash app gallery: <https://dash.gallery/Portal/>
- Dash community components: <https://community.plotly.com/t/community-components-index/60098>

Cheat Sheet: Plotly and Dash

Function	Description	Syntax	Example
Plotly Express			
<code>scatter</code>	Create a scatter plot	<code>px.scatter(dataframe, x=x_column, y=y_column)</code>	<code>px.scatter(df, x=age_array, y=income_array)</code>
<code>line</code>	Create a line plot	<code>px.line(x=x_column, y=y_column, title='title')</code>	<code>px.line(x=months_array, y=no_bicycle_sold_array)</code>
<code>bar</code>	Create a bar plot	<code>px.bar(x=x_column, y=y_column, title='title')</code>	<code>px.bar(x=grade_array, y=score_array, title='Pass Percentage')</code>
<code>sunburst</code>	Create a sunburst plot	<code>px.sunburst(dataframe, path=[col1,col2..], values='column', title='title')</code>	<code>px.sunburst(data, path=['Month', 'DestStateName'], values='Flights', title='Flight Distribution Hierarchy')</code>
<code>histogram</code>	Create a histogram	<code>px.histogram(x=x, title='title')</code>	<code>px.histogram(x=heights_array, title='Distribution of Heights')</code>
<code>bubble</code>	Create a bubble chart	<code>px.scatter(dataframe, x=x, y=y, size=size, title='title')</code>	<code>px.scatter(bub_data, x='City', y='Numberofcrimes', size='Numberofcrimes', hover_name='City', title='Crime Statistics')</code>
<code>pie</code>	Create a pie chart	<code>px.pie(values=x, names=y, title='title')</code>	<code>px.pie(values=xsp_percent, names=house_holdcategories, title='Household Expenditure')</code>
Plotly Graph Objects			
<code>Scatter</code>	Create a scatter	<code>go.Scatter(x=x, y=y, mode='markers')</code>	<code>go.Scatter(x=age_array, y=income_array, mode='markers')</code>
	Create a line plot	<code>go.Scatter(x=x, y=y, mode='lines')</code>	<code>go.Bar(x=months_array, y=no_bicycle_sold_array, mode='lines')</code>
<code>add_trace</code>	Add additional traces to an existing figure	<code>fig.add_trace(trace_object)</code>	<code>fig.add_trace(go.Scatter(x=months_array, y=no_bicycle_sold_array))</code>
<code>update_layout</code>	Update the layout of a figure, such as title, axis labels, and annotations.	<code>fig.update_layout(layout_object)</code>	<code>fig.update_layout(title='Bicycle Sales', xaxis_title='Months', yaxis_title='Number of Bicycles Sold')</code>
Dash			
<code>dash_core_components.Input</code>	Create an input component	<code>dcc.Input(value='', type='text')</code>	<code>dcc.Input(value='Hello', type='text')</code>
<code>dash_core_components.Graph</code>	Create a graph component	<code>dcc.Graph(figure=fig)</code>	<code>dcc.Graph(figure=fig)</code>
<code>dash_html_components.Div</code>	Create a div element	<code>html.Div(children=component_list)</code>	<code>html.Div(children=[html.H1('Hello Dash'), html.P('Welcome to Dash')])</code>
<code>dash_core_components.Dropdown</code>	Create a dropdown component	<code>dcc.Dropdown(options=options_list, value=default_value)</code>	<code>dcc.Dropdown(options=[{'label': 'Option 1', 'value': '1'}, {'label': 'Option 2', 'value': '2'}], value='1')</code>

Lab 4

- 1 Practice creating dashboards with Plotly:
`Plotly_Basics.ipynb`, data `airline_data.csv`
- 2 Practice creating dashboards with Dash:
`dash_interactivity.py` and `flight_delay.py`, data `airline_data.csv`
- 3 Practice creating dashboards with Plotly and Dash for your own research of interest by using your collected data